UTN · PROGRAMACIÓN ORIENTADA A OBJETOS

Cómo resolver un parcial de POO en Python

Una receta que se repite en todos los simulacros: cambian los nombres y las cuentas de negocio, pero los pasos son siempre los mismos.

Hospital | Mantenimiento | Empresa de Transporte | VS Code + pytest

0 Leer el enunciado
★ Leer el test primero
1 Armar las carpetas
2 Clases simples
3 Herencia
4 La clase contenedora
5 Leer el CSV
6 Correr los tests

Lo que NO cambia entre ejercicios

- Un archivo `.py` por clase
- Constructor, `__str__`, validaciones
- Base abstracta + hijas cuando hay "tipos"
- Una contenedora con una lista
- `main.py` que lee el CSV

Lo que SÍ cambia

- Qué sustantivos son las clases
- Qué atributos tiene cada una
- La cuenta de negocio puntual
- Cómo viene armado el CSV
- El estilo de nombres (camelCase / snake_case)

0 Leer el enunciado ANTES de escribir código

Una lectura completa, lápiz en mano. Anotá lo siguiente:

Lo que ves en el enunciado	Lo que significa	Ejemplo (Hospital)
Sustantivos que se repiten	Son las **clases**	"paciente", "atención médica", "hospital"
Los datos de cada uno	Son los **atributos**	un paciente tiene nombre, síntoma, habitual
"puede ser de tipo A o de tipo B"	Hay **herencia**	atención médica / atención de farmacia
"calcular", "sumar", "promediar", "cuántos..."	Son los **métodos**	`importeACobrar()`, promedio entre dos valores
Menciona un archivo .csv	Hay que leerlo en `main.py`	carpeta `data/` con 3 archivos

Ojo con el estilo de nombres

En Hospital todo se pidió en **camelCase** (`importeACobrar`, `addAtencion`). No des por sentado que siempre es así: algunos ejercicios piden **snake_case** (`calcular_total`, `agregar_item`). Fijate qué estilo usa el enunciado o el diagrama de *ese* ejercicio y mantenelo en todas las clases.

★ Si te dan un archivo de tests: leelo PRIMERO

Es el atajo más importante de toda la guía.

Por qué primero el test

Los tests ya te dicen **cómo se tiene que llamar todo**, sin ambigüedad. Armar las clases mirando solo el enunciado y adivinando nombres es la forma más común de perder tiempo corrigiendo `ImportError` y `AttributeError` después.

Dónde lo ves en el test	Qué te dice
`from atencion import Atencion, AtencionMedica` `from paciente import Paciente`	El nombre **exacto del archivo** .py y el nombre **exacto de cada clase** (con sus mayúsculas).
`Paciente("Ana", 1, True)`	El **orden y la cantidad de parámetros** del constructor.
`atencion.codigo` `atencion.tipoDeCobro = 2`	El nombre exacto de cada **atributo**.
`hospital.addAtencion(x)` `atencion.importeACobrar()`	El nombre exacto y los **argumentos de cada método**.
`assert ...importeACobrar() == 900.0`	Un **ejemplo ya resuelto**: sirve para verificar a mano tu cuenta antes de programarla.

Resumen: primero el test (si lo hay), después el enunciado para entender el *porqué* de cada cuenta.

1 Esqueleto de carpetas

Siempre igual. Un archivo por clase.

```
NombreDelEjercicio/
├── data/              # los .csv que te dieron
├── tests/             # el test del profe, si te lo dieron
├── main.py            # lee los CSV y arma todo
├── claseSimple1.py    # una clase por archivo
├── claseSimple2.py
├── claseBase.py       # base + hijas pueden vivir juntas
└── contenedora.py     # la que guarda la lista
```

Regla de nombres

Archivo en **minúscula**, clase adentro con **Mayúscula inicial**: `paciente.py` → `class Paciente:`. Si hay test, estos nombres salen directo de sus `import`; no los inventes.

2 Clases SIN herencia

Empezá por acá: son las más fáciles.

Molde general

```python
class NombreClase:
    def __init__(self, atributo1, atributo2, atributo3):
        # Las condiciones del enunciado ("debe ser positivo",
        # "solo puede ser 1, 2 o 3") van ACÁ, con raise ValueError
        self.atributo1 = atributo1
        self.atributo2 = atributo2
        self.atributo3 = atributo3

    def __str__(self):           # esto es el "toString"
        return f"NombreClase: {self.atributo1}, {self.atributo2}"
```

Ejemplo

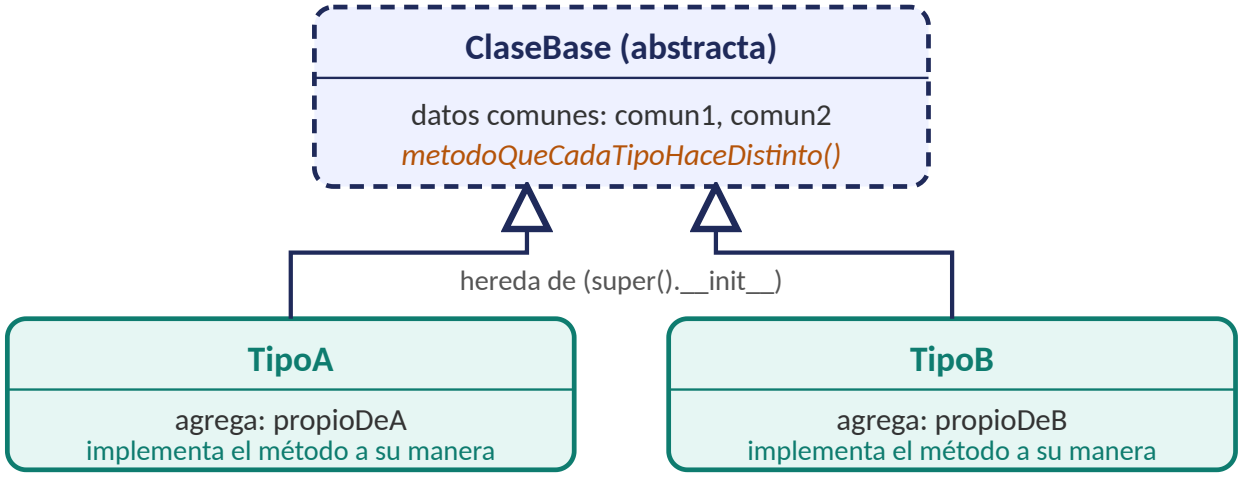
```python
class Persona:                   # "cada persona tiene nombre, edad y altura"
    def __init__(self, nombre, edad, altura):
        self.nombre = nombre
        self.edad = edad
        self.altura = altura
```

¿Y el "acceso y modificación" que pide el enunciado?

En Python no hace falta escribirlo aparte: al ser atributos públicos ya se lee con `objeto.atributo` y se modifica con `objeto.atributo = nuevoValor`. Getters y setters solo si el enunciado pide validar *también al modificar*.

3 Clases CON herencia

Cuando el enunciado dice «hay dos tipos de...»



La base guarda lo común; cada hija agrega lo suyo e implementa el método abstracto a su manera.

La base

```python
from abc import ABC, abstractmethod

class ClaseBase(ABC):                    # abstracta: nunca se instancia sola
    def __init__(self, comun1, comun2):
        self.comun1 = comun1
        self.comun2 = comun2

    @abstractmethod
    def metodoQueCadaTipoHaceDistinto(self):
        pass                             # cada hija lo completa
```

Cada hija

```python
class TipoA(ClaseBase):
    def __init__(self, comun1, comun2, propioDeA):
        super().__init__(comun1, comun2)   # arma la parte común
        self.propioDeA = propioDeA

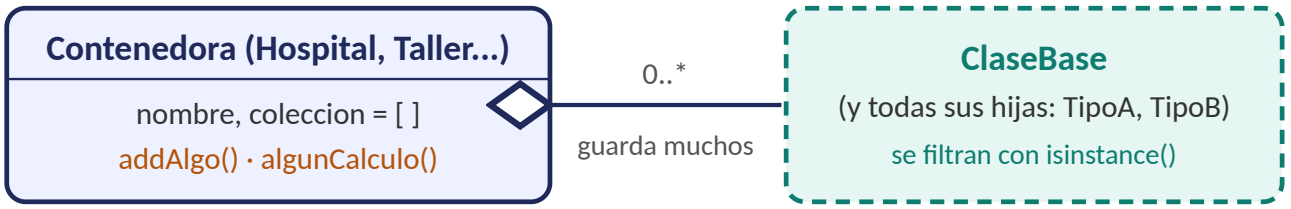
    def metodoQueCadaTipoHaceDistinto(self):
        ...                                  # la cuenta puntual de este tipo
```

Truco para encontrar el método abstracto

Es el cálculo que **da un resultado distinto según el tipo**. Ej.: «el importe se calcula distinto si es consulta médica o si es farmacia».

4 La clase contenedora

Hospital, Taller, Empresa… junta todo en una lista y hace las cuentas grupales.



Molde general

```python
class Contenedora:
    def __init__(self, nombre):
        self.nombre = nombre
        self.coleccion = []

    def addAlgo(self, objeto):
        self.coleccion.append(objeto)

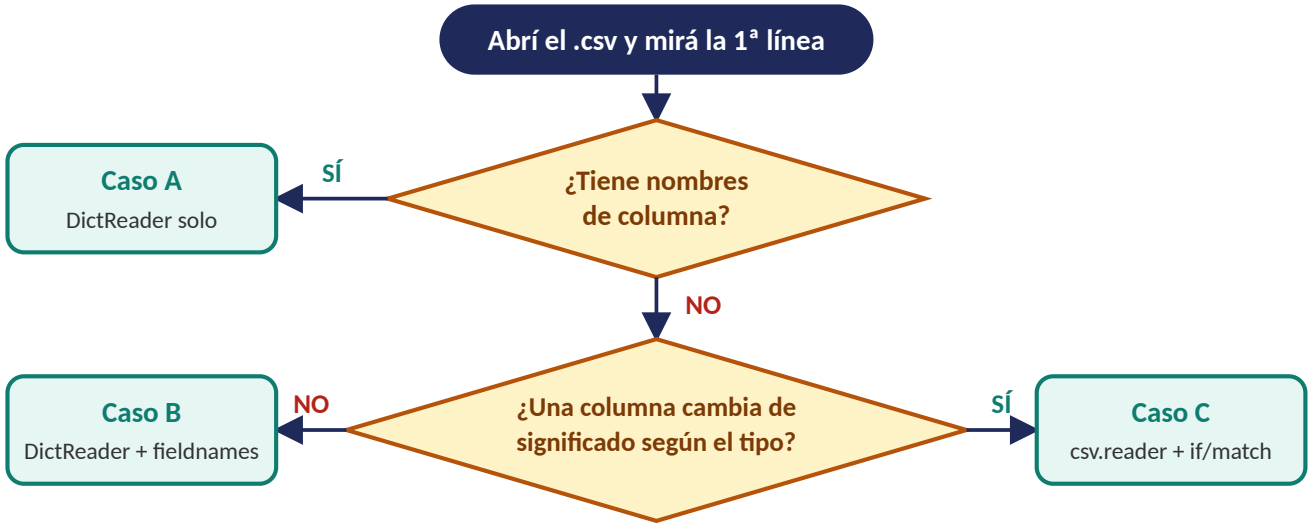
    def algunCalculo(self):
        return sum(o.valor for o in self.coleccion if isinstance(o, TipoA))
```

Listas con tipos mezclados

Si la colección mezcla tipos distintos, usá `isinstance(o, TipoA)` para quedarte solo con los que interesan en cada cuenta.

5 main.py: leer los CSV y armar los objetos

Antes de escribir código, abrí el .csv en un editor de texto y mirá la primera línea.



Tres casos posibles: cada uno se lee distinto.

Caso A El CSV tiene encabezado

pacientes.csv

```
codigo_atencion,nombre,sintoma,habitual
1,Juan Perez,2,true
2,Luis Gomez,1,false
```

`csv.DictReader(archivo)` **sin** `fieldnames` usa esa primera línea como claves del diccionario.

```python
with open("data/archivo.csv", newline="", encoding="utf-8") as archivo:
    for fila in csv.DictReader(archivo):
        codigo = int(fila["codigo_atencion"])
        ...
```

Caso B El CSV NO tiene encabezado (arranca directo con datos)

datos.csv

```
1,4/7,MARIA ROSARIO,15300,2,4700
2,4/7,JULIAN,11100,1,1400
```

Si acá usás `DictReader` a secas, Python toma la primera fila de **datos** como si fueran los nombres de columna: la perdés y te quedan claves raras como `fila["1"]`. Solución: pasale vos los nombres con `fieldnames`, en el mismo orden de las columnas.

```python
CAMPOS = ["tipo", "fecha", "operario", "importe_repuestos", "campo5", "campo6"]

with open("data/archivo.csv", newline="", encoding="utf-8") as archivo:
    for fila in csv.DictReader(archivo, fieldnames=CAMPOS):
        ...
```

Caso C Una misma columna significa cosas DISTINTAS según el tipo de fila

El enunciado dice algo como *«si es de tipo 1, la columna 5 es X; si es de tipo 2, la columna 5 es Y»*. Ejemplo inventado: vehículos, donde la columna 5 es «motivo de baja» (tipo 1) o «kilometraje» (tipo 2).

vehiculos.csv

```
1,ABC123,Ford,2020,fallo de motor
2,XYZ987,Fiat,2019,54000
```

`DictReader` con `fieldnames` fijo no sirve (el nombre de la columna 5 cambiaría según la fila). Se usa `csv.reader` —cada fila es una **lista**— y un `if` que decide qué es cada posición:

```python
with open("data/archivo.csv", newline="", encoding="utf-8") as archivo:
    for fila in csv.reader(archivo):
        tipo = int(fila[0])
        patente, marca, anio = fila[1], fila[2], int(fila[3])
        if tipo == 1:
            objeto = ClaseTipo1(patente, marca, anio, fila[4])          # motivo de baja
        elif tipo == 2:
            objeto = ClaseTipo2(patente, marca, anio, float(fila[4]))   # kilometraje
```

Es el mismo caso del Paso 3 (herencia): los dos tipos comparten las columnas base y cada uno agrega lo suyo; solo que la «rama según el tipo» ocurre al leer el CSV.

newline="" — SIEMPRE, tenga o no encabezado

Es independiente de los tres casos. La documentación oficial de `csv` lo recomienda porque el propio módulo interpreta los saltos de línea; si Python los traduce solo, en Windows puede meter `\r` de más o partir filas. Tomalo como norma fija:
`open("archivo.csv", newline="", encoding="utf-8")`

Situación	Qué usar	Cómo accedés al dato
CSV con encabezado	`csv.DictReader(archivo)`	`fila["nombre_columna"]` (fila = diccionario)
CSV sin encabezado	`csv.DictReader(archivo, fieldnames=CAMPOS)`	`fila["nombre_columna"]`
Columnas de significado variable	`csv.reader(archivo)`	`fila[0]`, `fila[1]` … (fila = lista)

Todo lo que viene de un CSV es texto (`str`): convertí con `int(...)` / `float(...)`, y los booleanos (“true”/“false”) con una función tipo `aBooleano()`.

Dos errores clásicos con `csv.DictReader`

Error 1 — usar fila[0] con DictReader → KeyError: 0

`DictReader` da cada fila como **diccionario** (clave → valor), no como lista. Un diccionario no tiene «posición 0»: busca una clave que literalmente sea el número 0, y no existe. El acceso posicional es de `csv.reader`. **No se mezclan:** `csv.reader` → lista → `fila[0]`; `DictReader` → diccionario → `fila["columna"]`.

Error 2 — pasar fieldnames cuando el archivo YA tiene encabezado

No explota ahí mismo, y eso es lo traicionero: `DictReader` usa tus claves para **todas** las filas desde la primera línea, así que el encabezado real se lee como un dato más. La primera iteración da `{"codigo_atencion": "codigo_atencion", ...}` y más adelante falla con `ValueError: invalid literal for int()`.

Cómo saltear el encabezado

```
# Si igual necesitás tus propios fieldnames en un archivo CON encabezado:
with open("data/pacientes.csv", newline="", encoding="utf-8") as archivo:
    next(archivo)                                    # descarta el encabezado real
    for fila in csv.DictReader(archivo, fieldnames=CAMPOS):
        ...
# Pero si tus nombres son los mismos que ya trae el archivo: NO pases fieldnames (Caso A).
```

6 Correr los tests y corregir de a un error por vez

Leé el primer error de arriba hacia abajo, arreglalo y volvé a correr.

Terminal de VS Code, parado en la carpeta del proyecto

```
python -m pytest tests/ -v
```

No leas todos los errores juntos

Suele ser el mismo error repetido en cascada: arreglás el primero y desaparecen varios.

pytest.ini — guardalo en la raíz del proyecto, al lado de main.py

```
[pytest]
testpaths = tests
python_files = test_*.py
addopts = -q
```

Con ese archivo, `pytest` ya sabe dónde buscar: alcanza con escribir `pytest` o `python -m pytest` a secas, sin repetir la ruta cada vez.

Errores de entorno más comunes

Ya nos pasaron en Hospital. Si ves esto, no es tu código.

Lo que ves	Qué pasa	Qué hacer
`pytest no se reconoce como nombre de un cmdlet…`	Windows no encuentra el comando `pytest` (PATH).	Usá `python -m pytest ...` en vez de `pytest ...`. Si falta instalarlo: `python -m pip install pytest`.
`ImportError: cannot import name X from X`	El archivo existe pero no tiene (o no ve) una clase con ese nombre exacto.	Revisá mayúsculas/minúsculas del nombre de la clase y **guardá el archivo** (`Ctrl+S`). Un punto blanco en la pestaña de VS Code = cambios sin guardar.
Corregiste algo y el error no cambia	Python/pytest están usando una versión vieja en caché.	Borrá las carpetas `__pycache__` y `.pytest_cache` y volvé a correr.

El caso del archivo vacío

En Hospital, el error persistió aun con el código perfecto en pantalla: `paciente.py` pesaba **0 bytes** en el disco porque nunca se había guardado. Ante un `ImportError` que no tiene sentido, mirá primero si el archivo está guardado. Para guardar todos los abiertos: `Ctrl+K` y luego `S`.

Rayitas rojas con «abc» al lado de las palabras

No son errores de código: es una extensión de corrector ortográfico (Code Spell Checker) marcando palabras en español. Se apaga desde Extensiones → buscar «spell» → **Disable**.

Checklist antes de dar el parcial por terminado

- [ ] Leí el **test primero** (si había) y los nombres de archivos, clases, atributos y métodos coinciden exactamente.
- [ ] Identifiqué las **clases**, los **atributos** y dónde hay **herencia** antes de escribir código.
- [ ] La clase base es **abstracta** y su método abstracto es el cálculo que cambia según el tipo.
- [ ] Cada constructor valida las condiciones del enunciado con `raise ValueError`.
- [ ] Cada clase tiene su `__str__` (el «toString» que pide el enunciado).
- [ ] Miré la primera línea del CSV y elegí el caso correcto (A, B o C); abrí siempre con `newline=""`.
- [ ] Convertí con `int()` / `float()` lo que sale del CSV como texto.
- [ ] Guardé todos los archivos (`Ctrl+K S`) y corrí `python -m pytest` hasta ver todo en verde.